LO3 EVIDENCE

Case Study 2 – Semester 2

FERNANDO RODRÍGUEZ MORAL
Fontys - ICT

Table of Contents

1. Introduction 3
1.1. Project Context 3
1.2. Relevance to This Learning Outcome 3
1.3. Requirements Addressed 4
2. Infrastructure as Code – Azure Bicep 5
2.1. Why Bicep over manual deployment 5
2.2. Main.bicep – Database tier 5
2.3. Private endpoint 7
2.4. Deployed infrastructure 8
3. App service deployment automation 9
3.1. Why automate app service via CLI 9
3.2. Azure CLI — Provisioning and Deployment 9
3.3. Application Settings — Secret Injection 10
3.4. Private Endpoint & DNS — App Service 11
4. Flask API — Application Layer Automation 12
4.1. Project Structure 12
4.2. App Factory — Automated Blueprint Registration 12
4.3. API Authentication — @require_api_key Decorator 13
4.4. Database Helper Functions 13
5. Monitoring script 15
5.1. Secure Credential Loading 15
5.2. Credential Validation at Startup 15
5.3. Metric Delivery — send_metric() 16
5.4. Batch Processing — send_all_metrics() 17
5.5. Execution modes 17
6. Automated Monitoring — Azure Monitor 18
6.1. Log Analytics Workspace 18

6.2. Automated metric alerts 18
7. Unit Testing 19
7.1. TestMetricsEndpoints 19
7.2. TestHealthEndpoints 20
8. Repeatable Deployment Pipeline 21
9. Reflection 22
9.1. What Went Well 22
9.2. Challenges Encountered 22

1. Introduction

1.1. Project Context

The Knowledge Hub library requires a centralized monitoring platform that collects telemetry from both on-premises servers (DC01, FS1, Monitoring VM) and Azure PaaS services, stores it in a cloud database, and exposes it through a REST API. The system consists of three automated components:

Component	Technology	Automation Scope
Database tier	Azure SQL + Bicep	Infrastructure provisioning, Private Endpoint, DNS
API tier	Flask + Azure App Service	App provisioning, deployment, secret injection
Collection tier	Python monitoring script	Metric collection, HTTP transmission, scheduling

1.2. Relevance to This Learning Outcome

This document demonstrates LO3 across all three automation dimensions defined by the learning outcome: **infrastructure** automation via Azure Bicep IaC, **application** automation via Azure CLI deployment scripts, and **business process** automation via a programmed Python monitoring loop. Together, these form a fully repeatable pipeline where no manual portal configuration is required after the initial setup.

1.3. Requirements Addressed

Requirement	Title	How Automation Addresses It
REQ-S2P1-08	Automated Task Execution	PowerShell scripts automate AD, DHCP, and GPO deployment; Python script automates monitoring
REQ-S2P1-07	Proactive System and Service Monitoring	Python script collects metrics, stores them in a database, and can query historical data for analysis
REQ-S2P1-01	Centralized User and Resource Management	PowerShell script builds the entire AD structure (OUs, users, groups) in one execution
REQ-S2P1-02	Structured and Segmented Network	PowerShell DHCP script automates scope configuration for each VLAN
REQ-S2P1-10	Standardized Workstation Management	GPO applied via automation ensures consistent workstation configuration across all machines

2. Infrastructure as Code – Azure Bicep

2.1. Why Bicep over manual deployment

The Azure SQL Database and Private Endpoint were deliberately deployed using Bicep rather than manual portal clicks. The justification is direct: a manual deployment cannot be reproduced identically, it leaves no audit trail, and it cannot be version-controlled. Bicep solves all three problems by expressing the desired infrastructure state as code:

2.2. Main.bicep – Database tier

The main.bicep file is the single source of truth for the database tier. It programmatically provisions the SQL Server and SQL Database with all security and cost settings enforced as code — not as post-deployment settings that could be missed:

```
@description('SQL Server admin username')
param adminLogin string = 'sqladmin'

@secure()
@description('SQL Server admin password')
param adminPassword string

@description('Location for all resources')
param location string = 'spaincentral'

@description('Environment tag')
param environment string = 'Dev'

resource sqlServer 'Microsoft.Sql/servers@2022-05-01-preview' = {
  name: 'sql-knowledgehub-dev'
  location: location
  tags: {
    Project: 'CS2-IN-NCB'
    Environment: environment
    Owner: 'mvandijk@knowledgehub.local'
    CostCenter: 'IT-Library'
    CreatedDate: '2026-04-10'
  }
  properties: {
    administratorLogin: adminLogin
    administratorLoginPassword: adminPassword
    publicNetworkAccess: 'Disabled'   // REQ-S2P2-03
    minimalTlsVersion: '1.2'
  }
}

resource sqlDatabase 'Microsoft.Sql/servers/databases@2022-05-01-preview' = {
  parent: sqlServer
  name: 'db-monitoring'
  location: location
  tags: {
    Project: 'CS2-IN-NCB'
    Environment: environment
    Owner: 'mvandijk@knowledgehub.local'
    CostCenter: 'IT-Library'
    CreatedDate: '2026-04-10'
  }
```

```
  sku: {
    name: 'GP_S_Gen5_1'
    tier: 'GeneralPurpose'
  }
  properties: {
    collation: 'SQL_Latin1_General_CP1_CI_AS'
    maxSizeBytes: 2147483648
    autoPauseDelay: 60          // Serverless: auto-pause after 60 min idle
    minCapacity: json('0.5')    // Minimum 0.5 vCores — cost control
    requestedBackupStorageRedundancy: 'Local'
  }
}

// Deny all public firewall rules — access via Private Endpoint only
resource firewallDenyAll 'Microsoft.Sql/servers/firewallRules@2022-05-01-preview' = {
  parent: sqlServer
  name: 'DenyAllPublicAccess'
  properties: {
    startIpAddress: '0.0.0.0'
    endIpAddress: '0.0.0.0'
  }
}

```

Key properties enforced by code:

- **publicNetworkAccess: 'Disabled'** — satisfies REQ-S2P2-03; no public internet route to the database at the infrastructure level.
- **minimalTlsVersion: '1.2'** — rejects all connections using TLS 1.0 or 1.1.
- **autoPauseDelay: 60** — serverless compute pauses after 60 minutes of inactivity; eliminates idle costs.
- **minCapacity: 0.5** — minimum vCore allocation; reduces cost during low-traffic periods.
- **Tags** — all five mandatory tags (Project, Environment, Owner, CostCenter, CreatedDate) applied at deployment, satisfying the Azure Policy Require-Tag-* enforcement.

2.3. Private endpoint

The Private Endpoint for Azure SQL is declared within the Bicep deployment, automating the secure binding of the database to the Virtual Network. This ensures the endpoint is created consistently with every deployment and never left as a manual follow-up step:

Resource	Value
Private Endpoint Name	pe-sql-knowledgehub-spoke1
Target Subnet	vnet-spoke1-app / DatabaseSubnet
Private IP	10.1.3.4
DNS A Record	sql-knowledgehub-dev → 10.1.3.4 (TTL 3600)
Private DNS Zone	privatelink.database.windows.net
VNet Links	Hub ✅ / Spoke1 ✅ / Spoke2 ✅
Provisioning State	Succeeded ✅
Connection State	Approved ✅

2.4. Deployed infrastructure

The following SQL resources were successfully provisioned via Bicep:

SQL Server:

Property	Value
Name	sql-knowledgehub-dev
Location	Spain Central
Version	12.0
Admin Login	sqladmin
Public Network Access	**Disabled**
Minimum TLS Version	1.2

SQL Database:

Property	Value
Name	db-monitoring
Status	Online ✅
Edition	Standard
Service Objective	S0
Max Size	2 GB
Collation	SQL_Latin1_General_CP1_CI_AS

3. App service deployment automation

3.1. Why automate app service via CLI

Using az webapp CLI commands instead of the Azure portal ensures that the App Service configuration — runtime version, startup command, and secrets — is captured in a reproducible script rather than in a portal session that cannot be replicated or audited.

3.2. Azure CLI — Provisioning and Deployment

The full App Service lifecycle is automated through three sequential CLI scripts:

Step 1 — Create the App Service:

```
az webapp create \
  --name app-knowledgehub-api \
  --resource-group rg-knowledgehub-spoke1 \
  --plan asp-knowledgehub-api \
  --runtime "PYTHON:3.12"

```

Step 2 — Configure the Gunicorn startup command:

```
az webapp config set \
  --name app-knowledgehub-api \
  --resource-group rg-knowledgehub-spoke1 \
  --startup-file "gunicorn --bind 0.0.0.0:8000 app:app"

```

Step 3 — Deploy the Flask application code:

```
az webapp up \
  --name app-monitoring-knowledgehub \
  --resource-group rg-knowledgehub-spoke1 \
  --runtime "PYTHON:3.11"

```

Deployed App Service — Proof:

Property	Value
Name	app-monitoring-knowledgehub
Location	Spain Central
State	**Running** ✅
App Service Plan	asp-monitoring-api — Basic B1
Default Hostname	app-monitoring-knowledgehub.azurewebsites.net

3.3. Application Settings — Secret Injection

Rather than hardcoding secrets in source files, all sensitive configuration is injected into the App Service as Application Settings via the CLI. The App Service then exposes these as environment variables at runtime, which the Flask Config class reads via os.getenv():

```
az webapp config appsettings set \
  --name app-monitoring-knowledgehub \
  --resource-group rg-knowledgehub-spoke1 \
  --settings \
    API_KEY="<secret>" \
    DB_CONNECTION_STRING="Driver={ODBC Driver 18 for SQL Server};\
      Server=tcp:sql-knowledgehub-dev.database.windows.net,1433;\
      Database=db-monitoring;Uid=sqladmin;Pwd=<secret>;\
      Encrypt=yes;TrustServerCertificate=no"

```

The Flask configuration class reads these at startup:

```
class Config:
    API_KEY            = os.getenv("API_KEY")
    DB_CONNECTION_STRING = os.getenv("DB_CONNECTION_STRING")
    DEBUG              = os.getenv("FLASK_DEBUG", "false").lower() == "true"
    HOST               = os.getenv("FLASK_HOST", "0.0.0.0")
    PORT               = int(os.getenv("FLASK_PORT", 5000))

```

A .env.example file is committed to Git as a template showing the required variable names without values, so any new developer knows exactly what to configure.

3.4. Private Endpoint & DNS — App Service

The Private Endpoint for the App Service was deployed and confirmed:

Property	Value
Name	pe-app-monitoring
Subnet	vnet-spoke1-app / AppSubnet
Private IP	10.1.1.4
DNS A Record	app-monitoring-knowledgehub → 10.1.1.4 (TTL 3600)
Private DNS Zone	privatelink.azurewebsites.net
VNet Links	Hub / Spoke1
Provisioning State	Succeeded
Connection State	Approved

4. Flask API — Application Layer Automation

4.1. Project Structure

The Flask API follows a modular structure designed for automated deployment and testability. Each concern is isolated in its own file, making the application independently deployable and unit-testable without external dependencies:

```
monitoring-api/
│
├── app/
│   ├── __init__.py       # Flask app factory — creates app, registers blueprints
│   ├── config.py         # Environment-based configuration (reads App Settings)
│   ├── models.py         # Data models: MonitoringMetric, AzurePaaSHealth
│   ├── auth.py           # @require_api_key decorator
│   ├── database.py       # pyodbc helpers: insert_metric, insert_health, get_metrics
│   └── routes/
│       ├── metrics.py    # POST/GET /api/v1/metrics
│       └── health.py     # POST /api/v1/health, GET /api/v1/health/status
│
├── tests/
│   ├── test_metrics.py   # TestMetricsEndpoints unit tests
│   └── test_health.py    # TestHealthEndpoints unit tests
│
├── .env                  # Local dev secrets — never committed to Git
├── .env.example          # Template committed to Git
├── .gitignore            # Excludes .env, __pycache__, etc.
├── requirements.txt      # Python dependencies
├── app.py                # Entry point
└── Dockerfile            # Optional: Container Apps deployment

```

4.2. App Factory — Automated Blueprint Registration

The app factory pattern in __init__.py automates blueprint registration and logging setup. A single create_app() call produces a fully configured, production-ready Flask instance:

```python
import logging
from flask import Flask
from app.config import Config

def create_app(config_class=Config):
    app = Flask(__name__)
    app.config.from_object(config_class)

    logging.basicConfig(
        level=logging.INFO,
        format="%(asctime)s | %(levelname)s | %(name)s | %(message)s"
    )

    from app.routes.metrics import metrics_bp
    from app.routes.health import health_bp
    app.register_blueprint(metrics_bp, url_prefix="/api/v1")
    app.register_blueprint(health_bp, url_prefix="/api/v1")

    return app

```

4.3. API Authentication — @require_api_key Decorator

All endpoints (except the public health status check) are protected by a reusable decorator that reads the key from the environment, never from code. Every unauthorized attempt is logged with the source IP:

```
def require_api_key(f):
    @wraps(f)
    def decorated(*args, **kwargs):
        api_key = request.headers.get("X-API-Key")
        if not api_key:
            logger.warning(f"AUTH | Missing API key | IP: {request.remote_addr}")
            return jsonify({"error": "Missing API key"}), 401
        if api_key != os.getenv("API_KEY"):
            logger.warning(f"AUTH | Invalid API key | IP: {request.remote_addr}")
            return jsonify({"error": "Invalid API key"}), 403
        return f(*args, **kwargs)
    return decorated

```

4.4. Database Helper Functions

All Azure SQL operations are encapsulated in typed helper functions using pyodbc with parameterized queries — protecting against SQL injection and providing clean separation between routing and data persistence:

```
def get_connection() -> pyodbc.Connection:
    conn_str = os.getenv("DB_CONNECTION_STRING")
    if not conn_str:
        raise ValueError("DB_CONNECTION_STRING environment variable is not set")
    return pyodbc.connect(conn_str)

def insert_metric(source_host: str, source_type: str, metric_name: str,
                  metric_value: float, unit: Optional[str], status: Optional[str]) -> None:
    sql = """
        INSERT INTO MonitoringMetrics
            (source_host, source_type, metric_name, metric_value, unit, status)
        VALUES (?, ?, ?, ?, ?, ?)
    """
    try:
        with get_connection() as conn:
            conn.execute(sql, (source_host, source_type, metric_name,
                               metric_value, unit, status))
            conn.commit()
            logger.info(f"DB - Metric inserted: {source_host} | {metric_name}={metric_value}")
    except pyodbc.Error as e:
        logger.error(f"DB - Failed to insert metric: {e}")
        raise

def insert_health(service_name: str, status: str,
                  response_ms: Optional[int], error_message: Optional[str]) -> None:
    sql = """
        INSERT INTO AzurePaaSHealth (service_name, status, response_ms, error_message)
        VALUES (?, ?, ?, ?)
    """
    try:
        with get_connection() as conn:
            conn.execute(sql, (service_name, status, response_ms, error_message))
            conn.commit()
```

```
            logger.info(f"DB - Health inserted: {service_name} | {status}")
    except pyodbc.Error as e:
        logger.error(f"DB - Failed to insert health record: {e}")
        raise
The get_metrics() query function supports optional filters and pagination:
python
def get_metrics(source_host=None, metric_name=None, limit=100) -> list:
    sql = """SELECT id, timestamp, source_host, source_type,
                    metric_name, metric_value, unit, status
             FROM MonitoringMetrics WHERE 1=1"""
    params = []
    if source_host:
        sql += " AND source_host = ?"
        params.append(source_host)
    if metric_name:
        sql += " AND metric_name = ?"
        params.append(metric_name)
    sql += " ORDER BY timestamp DESC OFFSET 0 ROWS FETCH NEXT ? ROWS ONLY"
    params.append(limit)
    with get_connection() as conn:
        cursor = conn.execute(sql, params)
        columns = [col[0] for col in cursor.description]
        return [dict(zip(columns, row)) for row in cursor.fetchall()]

```

4.5 API Design Specification

Base URL (accessible only via Private Endpoint — REQ-S2P2-03):

```
https://app-monitoring-knowledgehub.privatelink.azurewebsites.net/api/v1
```

Method	Endpoint	Auth	Description
POST	/api/v1/metrics	Required	Submit a monitoring metric
GET	/api/v1/metrics	Required	Retrieve stored metrics
POST	/api/v1/health	Required	Submit a PaaS health check
GET	/api/v1/health/status	**Public**	Confirm API is running

5. Monitoring script

5.1. Secure Credential Loading

The script uses python-dotenv to load MONITOR_API_URL and MONITOR_API_KEY from the OS environment or a local .env file at startup — preventing secrets from ever appearing in source code:

```
from dotenv import load_dotenv
load_dotenv()

API_URL = os.getenv("MONITOR_API_URL")  # e.g. https://app-knowledgehub-api.azurewebsites.net
API_KEY = os.getenv("MONITOR_API_KEY")  # same key configured on the App Service

```

5.2. Credential Validation at Startup

Before executing any collection logic, the script validates all required variables and exits cleanly with a clear error message if any are missing — preventing silent runtime failures mid-cycle:

```
def main():
    args = parse_args()
    if not args.dry_run:
        if not API_URL:
            logger.error("MONITOR_API_URL not set. Add it to .env or set env variable.")
            sys.exit(1)
        if not API_KEY:
            logger.error("MONITOR_API_KEY not set. Add it to .env or set env variable.")
            sys.exit(1)
    logger.info(f"Target API: {API_URL}")

```

5.3. Metric Delivery — send_metric()

The send_metric() function handles the full HTTP POST lifecycle: header injection, payload serialization, timeout enforcement, and per-exception error handling. Network failures never crash the main loop:

```
def send_metric(metric: dict) -> bool:
    endpoint = f"{API_URL}/api/v1/metrics"
    headers = {
        "Content-Type": "application/json",
        "X-API-Key": API_KEY
    }
    try:
        response = requests.post(endpoint, json=metric, headers=headers, timeout=10)
        if response.status_code == 201:
            logger.info(f"SENT: {metric['metric_name']}={metric['metric_value']} | {response.status_code}")
            return True
        else:
            logger.warning(f"FAIL: {metric['metric_name']} | {response.status_code} {response.text}")
            return False
    except requests.exceptions.ConnectionError:
        logger.error("CONN ERROR: Cannot reach endpoint")
        return False
    except requests.exceptions.Timeout:
        logger.error("TIMEOUT: Endpoint did not respond within 10s")
        return False
    except requests.exceptions.RequestException as e:
        logger.error(f"REQUEST ERROR: {e}")
        return False

```

5.4. Batch Processing — send_all_metrics()

At the end of each collection cycle, the function aggregates all results and returns an automated summary report — giving operators immediate visibility into collection health without manual log parsing:

```
def send_all_metrics(metrics: list[dict]) -> dict:
    """Send all collected metrics and return a summary report."""
    sent = failed = 0
    for metric in metrics:
        if send_metric(metric):
            sent += 1
        else:
            failed += 1
    return {"sent": sent, "failed": failed, "total": len(metrics)}

```

5.5. Execution modes

A CLI interface provides three execution modes covering all operational scenarios:

Mode	Command	Use Case
Single run	python monitor.py	One-off collection — testing or on-demand snapshot
Continuous	python monitor.py --interval 60	Production — loops indefinitely, sleeps 60s between cycles
Dry run	python monitor.py --dry-run	Validates collection logic locally without sending data to the API

The --dry-run mode is particularly valuable for CI validation: the full collection pipeline executes and logs output, but no HTTP requests are made, so the test can run in isolation without a live VPN connection to Azure.

6. Automated Monitoring — Azure Monitor

6.1. Log Analytics Workspace

All App Service telemetry is forwarded automatically to the Log Analytics workspace log-knowledgehub-monitoring via the diagnostic setting diag-appservice. This provides a permanent, queryable audit trail for all API activity:

Log Category	Enabled	Purpose
HTTP Logs	✅	Every API request — method, path, status, source IP
App Logs	✅	Flask structured logs (AUTH, METRIC, HEALTH, DB prefixes)
Console Logs	✅	Runtime errors and stack traces
Audit Logs	✅	Platform-level configuration changes
AllMetrics	✅	CPU, memory, request count, response times

6.2. Automated metric alerts

Two alerts fire automatically without manual intervention:

Alert	Metric	Condition	Severity	Action
alert-appservice-5xx	Http5xx	> 5 errors / 5 min	1 — Critical	Email to mvandijk@knowledgehub.local
alert-sql-cpu-high	cpu_percent	> 80% avg / 15 min	2 — Warning	Email to mvandijk@knowledgehub.local

Both route through the action group ag-knowledgehub-alerts (Group: KHAlerts, Common Alert Schema: Enabled), providing a standardized notification format that can be extended to webhooks, SMS, or ITSM integrations in the future.

7. Unit Testing

Automated tests validate all API endpoint behaviors before deployment, catching regressions at development time. The test suite uses Python's unittest framework with unittest.mock to isolate the database layer.

7.1. TestMetricsEndpoints

```
class TestMetricsEndpoints(unittest.TestCase):

    def setUp(self):
        self.app = create_app(Config)
        self.app.config["TESTING"] = True
        self.client = self.app.test_client()
        self.headers = {"X-API-Key": "test-key", "Content-Type": "application/json"}

    @patch("os.getenv", side_effect=lambda k, *a: "test-key" if k == "API_KEY" else None)
    @patch("app.database.insert_metric")
    def test_post_metric_success(self, mock_insert, mock_env):
        """POST /api/v1/metrics with valid data returns 201."""
        payload = {"source_host": "DC01", "source_type": "onprem",
                   "metric_name": "cpu_percent", "metric_value": 47.3,
                   "unit": "%", "status": "OK"}
        response = self.client.post("/api/v1/metrics",
                                    data=json.dumps(payload),
                                    headers=self.headers)
        self.assertEqual(response.status_code, 201)
        self.assertEqual(response.get_json()["data"]["source_host"], "DC01")

    @patch("os.getenv", side_effect=lambda k, *a: "test-key" if k == "API_KEY" else None)
    def test_post_metric_missing_field(self, mock_env):
        """POST /api/v1/metrics with missing field returns 400."""
        response = self.client.post("/api/v1/metrics",
                                    data=json.dumps({"source_host": "DC01"}),
                                    headers=self.headers)
        self.assertEqual(response.status_code, 400)

    def test_post_metric_no_api_key(self):
        """POST /api/v1/metrics without API key returns 401."""
        response = self.client.post("/api/v1/metrics",
                                    data=json.dumps({}),
                                    headers={"Content-Type": "application/json"})
        self.assertEqual(response.status_code, 401)

    @patch("os.getenv", side_effect=lambda k, *a: "test-key" if k == "API_KEY" else None)
    @patch("app.database.get_metrics", return_value=[])
    def test_get_metrics_success(self, mock_get, mock_env):
        """GET /api/v1/metrics returns 200 with results list."""
        response = self.client.get("/api/v1/metrics", headers=self.headers)
        self.assertEqual(response.status_code, 200)
        self.assertIn("results", response.get_json())

```

7.2. TestHealthEndpoints

```
class TestHealthEndpoints(unittest.TestCase):

    def test_api_status_public(self):
        """GET /api/v1/health/status is public and returns 200."""
        response = self.client.get("/api/v1/health/status")
        self.assertEqual(response.status_code, 200)
        self.assertEqual(response.get_json()["status"], "ok")
        self.assertEqual(response.get_json()["service"], "monitoring-api")

    @patch("os.getenv", side_effect=lambda k, *a: "test-key" if k == "API_KEY" else None)
    @patch("app.database.insert_health")
    def test_post_health_success(self, mock_insert, mock_env):
        """POST /api/v1/health with valid data returns 201."""
        payload = {"service_name": "AzureSQL-DB", "status": "Healthy",
                   "response_ms": 42, "error_message": None}
        response = self.client.post("/api/v1/health",
                                    data=json.dumps(payload),
                                    headers=self.headers)
        self.assertEqual(response.status_code, 201)
        self.assertEqual(response.get_json()["data"]["service_name"], "AzureSQL-DB")

    @patch("os.getenv", side_effect=lambda k, *a: "test-key" if k == "API_KEY" else None)
    def test_post_health_missing_field(self, mock_env):
        """POST /api/v1/health with missing field returns 400."""
        response = self.client.post("/api/v1/health",
                                    data=json.dumps({"service_name": "AzureSQL-DB"}),
                                    headers=self.headers)
        self.assertEqual(response.status_code, 400)

    def test_post_health_no_api_key(self):
        """POST /api/v1/health without API key returns 401."""
        response = self.client.post("/api/v1/health",
                                    data=json.dumps({}),
                                    headers={"Content-Type": "application/json"})
        self.assertEqual(response.status_code, 401)
```

8. Repeatable Deployment Pipeline

Because the entire environment is defined in version-controlled code and scripts, the full infrastructure and application stack can be torn down and rebuilt identically by any team member running five commands:

```
# 1. Deploy database tier (SQL Server + DB + Private Endpoint)
az deployment group create \
  --resource-group rg-knowledgehub-spoke1 \
  --template-file main.bicep \
  --parameters adminPassword="<secret>"

# 2. Create App Service
az webapp create \
  --name app-knowledgehub-api \
  --resource-group rg-knowledgehub-spoke1 \
  --plan asp-knowledgehub-api \
  --runtime "PYTHON:3.12"

# 3. Set startup command
az webapp config set \
  --name app-knowledgehub-api \
  --resource-group rg-knowledgehub-spoke1 \
  --startup-file "gunicorn --bind 0.0.0.0:8000 app:app"

# 4. Deploy Flask code
az webapp up \
  --name app-monitoring-knowledgehub \
  --resource-group rg-knowledgehub-spoke1

# 5. Inject secrets
az webapp config appsettings set \
  --name app-monitoring-knowledgehub \
  --resource-group rg-knowledgehub-spoke1 \
  --settings API_KEY="<secret>" DB_CONNECTION_STRING="<connstr>"

```

No manual portal configuration is required. Every security setting, tag, and connection is either declared in Bicep or injected via CLI — giving the team a single, auditable source of truth for the entire deployment.

9. Reflection

9.1. What Went Well

Bicep enforced security at deployment time, not after. Declaring publicNetworkAccess: 'Disabled' and minimalTlsVersion: '1.2' directly in the template meant these settings could never be accidentally omitted during a redeployment. In a manual portal workflow, security properties are easy to forget when rebuilding an environment under time pressure — Bicep removes that risk entirely.

Unit tests caught a real validation gap before deployment. The test_post_metric_missing_field test caught a case where the route handler raised an unhandled KeyError instead of returning a clean 400 Bad Request when source_type was absent from the payload. This would have produced a 500 Internal Server Error in production — the test caught it at local development time and the from_dict() method was hardened before deployment.

9.2. Challenges Encountered

Gunicorn startup command parsing in Azure App Service. Configuring the Linux App Service to serve Flask via Gunicorn was more involved than expected. The az webapp config set --startup-file command accepted the value without error, but on the first deployment the application failed to start. Diagnosing the issue required enabling App Service log streaming (az webapp log tail) and reading the Gunicorn binding error in the output — the port binding format needed to be 0.0.0.0:8000 rather than 0.0.0.0 alone. This highlighted that Azure App Service Linux containers have specific expectations about startup command format that are not well-surfaced in the portal UI.

Private Endpoint locking out local development. After the Bicep deployment set publicNetworkAccess: 'Disabled', all direct connections from a local development machine to Azure SQL over the public internet were immediately rejected — as intended by REQ-S2P2-03. This also blocked local unit test runs that used a live database connection. The solution was to restructure the test suite to mock the database layer (@patch("app.database.insert_metric")) for local tests, reserving live integration tests for the VPN-connected environment only. This was the right architectural decision — unit tests should not depend on live external services — but it required a refactor of the initial test setup.